

Invalid Action Masking

A Closer Look at Invalid Action Masking in Policy Gradient Algorithms

Shengyi Huang, Santiago Ontañón (2022)

현재 상태에서 실행할 수 없는 행동을 policy 분포 안에서 제외하는 방법

Invalid action의 정의

실행할 수 없는 행동과 실행 가능하지만 보상이 낮은 행동은 학습 신호가 다르다

행동 종류	환경에서 실행	결과 관측	처리
현재 상태에서 invalid	불가능	정상 결과가 없음	규칙이 알려 주면 mask 가능
Valid하지만 좋지 않음	가능	낮은 reward를 관측	Policy가 결과를 학습
상태 의존성	같은 index도 상태에 따라 달라짐	A_valid(s)가 변함	매 상태에서 validity 갱신

Mask는 싫은 행동이 아니라 현재 규칙상 불가능한 행동의 support를 제거한다

고정된 행동 목록의 탐색 낭비

Policy head는 전체 행동 index를 출력하지만 한 상태에서 가능한 subset은 작을 수 있다

구성	전체 policy	현재 상태
행동 index	항상 같은 전체 목록	일부 index만 실행 가능
Softmax support	Mask가 없으면 모든 index에 확률	Invalid index도 표본화될 수 있음
탐색 비용	불가능한 선택에도 draw를 사용	Reward를 얻는 valid sequence가 늦어짐
규칙 정보	Policy가 추론하기 전에도 환경이 보유	표본화 전에 적용 가능

알려진 feasibility를 sampling 전에 반영하면 invalid draw에 쓰는 탐색을 줄일 수 있다

μRTS 자원 수집 환경

관측은 27개 feature plane이고 행동은 여덟 component로 구성된다

Action component	범위	역할
Source Unit	0부터 hw- 1	명령을 받을 unit 위치
Action Type	6종	noop, 이동, 수집, 반환, 생산, 공격
방향 parameter 4종	각 4방향	이동, 수집, 반환, 생산 방향
Produce Type	7종	생산할 unit이나 building 종류
Attack Target Unit	0부터 hw- 1	공격 대상 위치

Map 크기와 valid source 확률

보통 선택 가능한 source가 base와 worker 두 개라면 균등 선택의 성공 확률이 급격히 낮아진다

Map	Source index 수	Valid source	균등 선택 확률
4 × 4	16	2	$2 / 16 = 0.125$
24 × 24	576	2	$2 / 576 \approx 0.0034$
크기 변화	36배	같음	약 36분의 1
의미	Invalid 후보가 증가	규칙상 가능한 unit은 적음	첫 reward 탐색이 어려워질 수 있음

같은 두 unit을 찾더라도 24 × 24에서는 random source 선택의 99% 이상이 invalid다

Policy gradient의 기본 경로

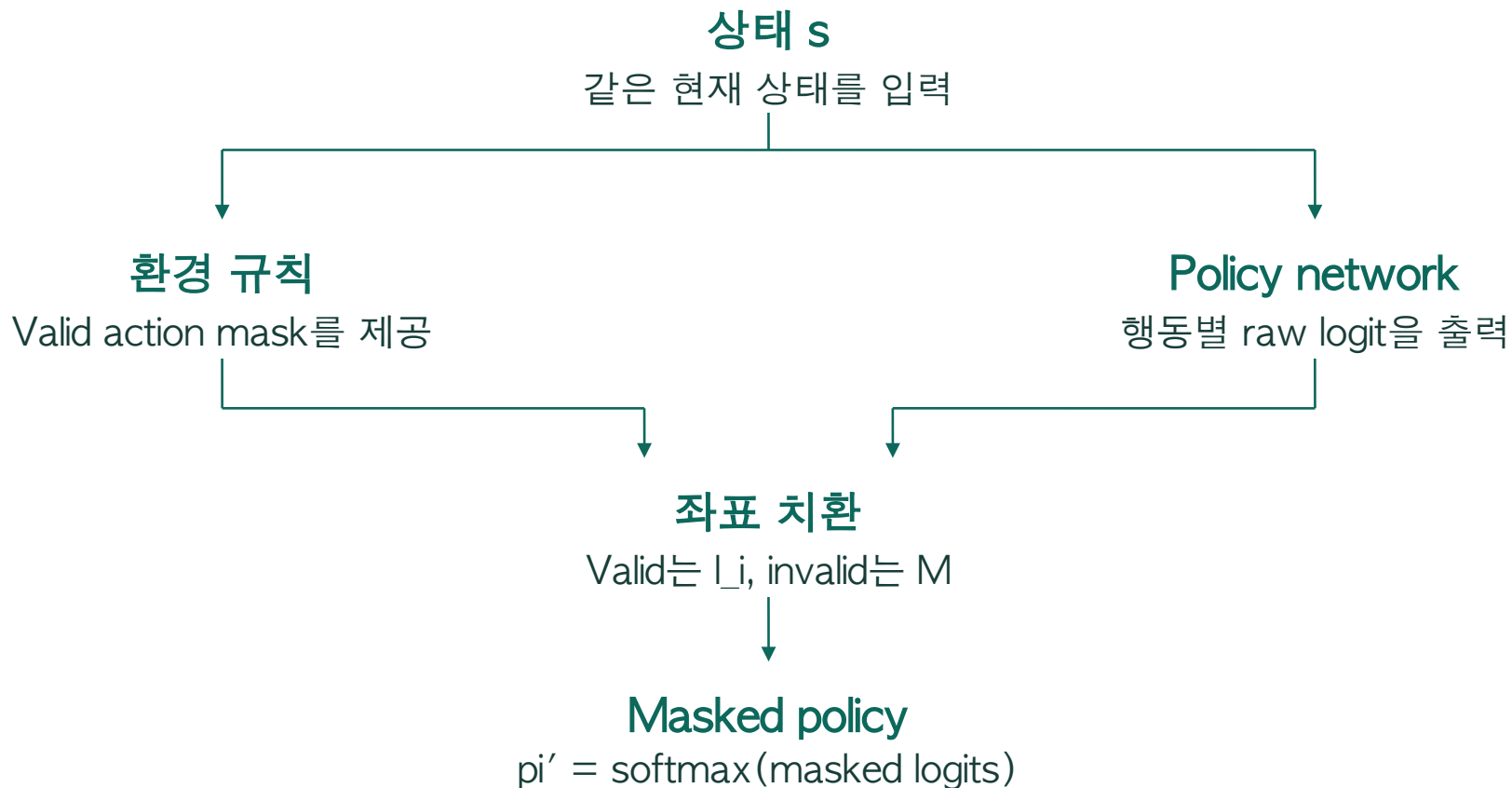
Mask가 없으면 network logit의 softmax가 sampling과 log-probability를 함께 정의한다

01	상태 s	Policy network에 현재 상태를 입력한다
↓		
02	Raw logits l	행동 index마다 실수 점수를 출력한다
↓		
03	Softmax π	Logit을 categorical 확률로 정규화한다
↓		
04	행동 표본	π 에서 a_t 를 뽑아 환경에 전달한다
↓		
05	Score update	$\log \pi(a_t s_t)$ 의 gradient에 G_t 를 곱한다

G_t 는 표본 행동 이후의 return 또는 advantage 계수이며 policy의 방향과 크기를 조절한다

Validity와 logit의 결합

환경 규칙과 neural logits는 병렬 입력이며 invalid 좌표를 상수 M으로 바꾼다



Masked softmax 수치 계산

교육용 계산에서는 $-\infty$ 극한을 사용해 두 invalid 행동의 확률을 정확한 0으로 표시한다

항목	a0	a1	a2	a3
Raw logit	0	1	2	3
현재 valid	yes	no	yes	no
Masked logit	0	$-\infty$	2	$-\infty$
exp 값	1	0	7.389	0
Masked pi'	0.1192	0	0.8808	0

Masked policy의 score gradient

a2를 뽑았을 때 finite M에서도 invalid raw-logit 좌표의 gradient는 정확히 0이다

항목	a0	a1	a2	a3
Masked π'	0.1192	0	0.8808	0
one-hot(a2)	0	0	1	0
$-\infty$ 극한 gradient	-0.1192	0	0.1192	0
Finite M gradient	$-\pi'_0$	0	$1-\pi'_2$	0
Raw-logit Jacobian	identity	0	identity	0

Proposition 1의 결론

상태별 identity-or-constant mask를 사용하면 update는 masked policy π' 의 policy gradient다

01	상태별 valid 집합	현재 규칙이 각 행동의 validity를 정한다
↓		
02	좌표별 mapping	Valid logit은 identity, invalid logit은 상수 M
↓		
03	미분 가능한 π'	Masked logits에 softmax를 적용한다
↓		
04	같은 π' 사용	Sampling과 log-probability를 모두 π' 에서 계산
↓		
05	Policy gradient	Update는 behavior policy π' 의 목적함수 gradient

정당화 대상은 unmasked π 가 아니라 실제 trajectory를 만드는 masked policy π' 다

Proposition 1의 적용 조건

논문의 증명은 특정 state-dependent mask를 다루며 임의의 pruning 방법을 포괄하지 않는다

조건 또는 범위	논문이 다룸	추가 검증이 필요한 경우
Validity 출처	현재 상태와 환경 규칙	Parameter에 의존하는 learned mask
좌표 mapping	Identity 또는 상수 M	불연속 pruning이나 다른 변환
Policy 구성	미분 가능한 logits와 softmax	다른 action interface
정확성	주어진 mask를 그대로 사용	오류가 있는 feasibility oracle
배포	Mask가 있는 π' 의 update	학습 뒤 mask를 제거한 policy

Mask와 invalid-action penalty

Mask는 sampling 전에 분포를 바꾸고 penalty는 invalid 선택 이후 return을 바꾼다

시점	Mask-consistent	Penalty
상태 입력	규칙에서 valid subset 확인	원래 policy를 그대로 사용
Sampling 전	Invalid logit을 M으로 치환	Invalid 행동도 확률을 가짐
행동 선택	π' 에서 feasible 행동 표본	π 에서 invalid 행동을 뽑을 수 있음
환경 결과	선택을 실행하고 정상 reward 관측	Invalid 선택에 r_{invalid} 를 추가
조정 항목	Validity 규칙의 정확성	Penalty 크기와 reward scale

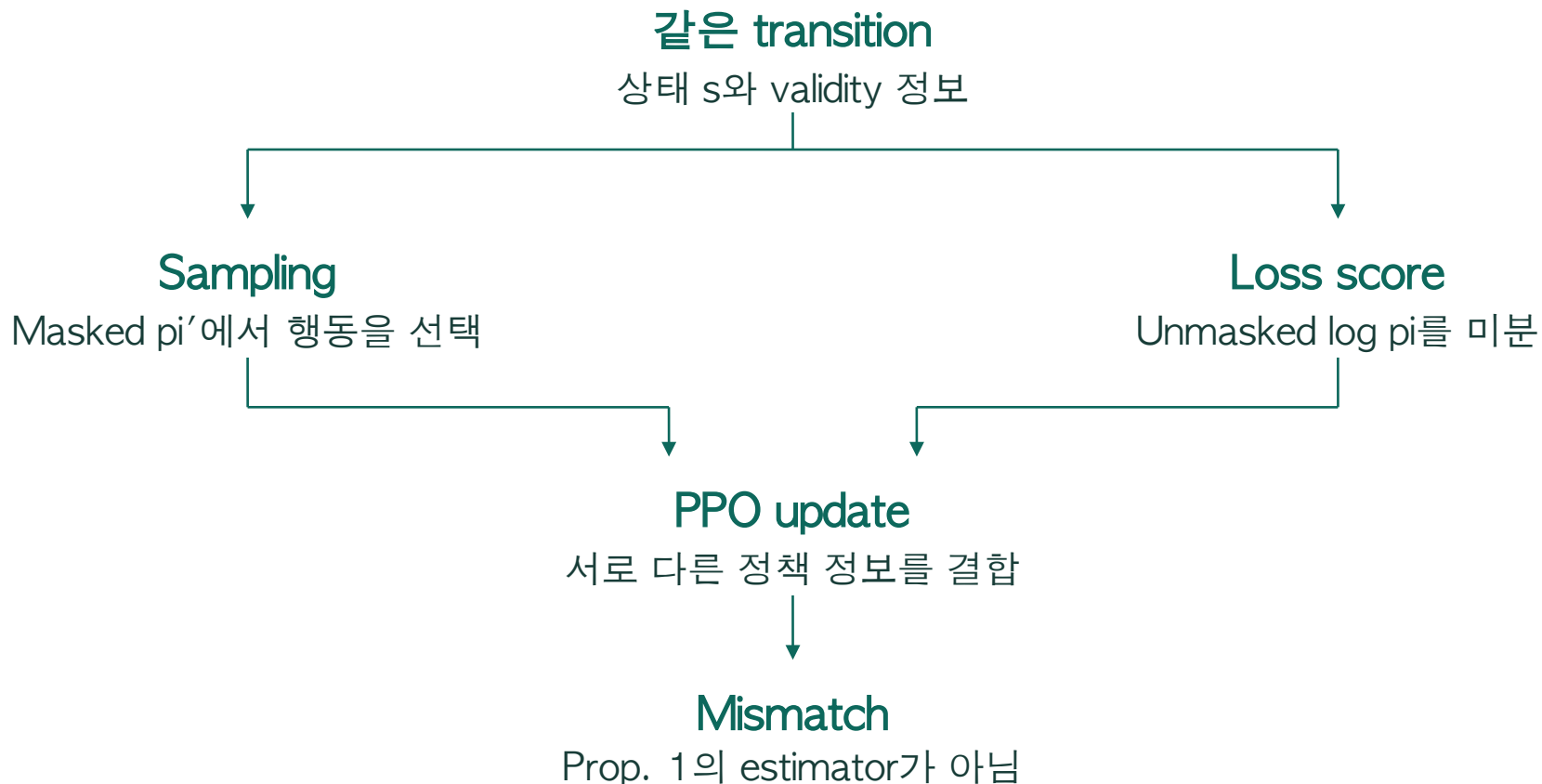
Naive masking의 gradient 불일치

행동은 π' 에서 뽑고 score는 unmasked π 로 계산하면 invalid logit에도 update가 생긴다

항목	a0	a1	a2	a3
Sampling π'	0.1192	0	0.8808	0
Unmasked π	0.0321	0.0871	0.2369	0.6439
a2 naive gradient	-0.0321	-0.0871	0.7631	-0.6439
Mask-consistent	-0.1192	0	0.1192	0
Invalid 좌표	valid	nonzero update	valid	nonzero update

Naive estimator의 두 분포

Sampling support와 loss의 score distribution이 달라 높은 return만으로 정당성을 확인할 수 없다



네 가지 실험 전략

Sampling distribution, loss distribution, evaluation mask를 구분하면 방법 차이가 드러난다

전략	Sampling	Loss score	평가 mask
Invalid penalty	Unmasked p_i	Unmasked p_i	없음
Mask-consistent	Masked p_i'	Masked p_i'	사용
Naive masking	Masked p_i'	Unmasked p_i	사용
Masking removed	학습은 masked p_i'	학습은 masked p_i'	제거
실제 mask 범위	Source와 Attack Target	같은 두 component	전략에 따라 다름

실험 지표와 집계 조건

Table 2는 네 seed의 평균이고 모든 시간 지표는 500,000 training step의 비율이다

지표	정의	방향
r_episode	마지막 10 episode의 평균 return	높을수록 좋음
t_first	첫 positive reward까지의 training-step 비율	낮을수록 좋음
t_solve	10-episode moving average가 40 초과까지의 비율	낮을수록 좋음
Table 2	4 random seed 평균	환경별 수치 비교
Figure curve	Weight 0.9로 smoothing, 1 표준편차 음영	Raw table 값과 구분

Mask-consistent 결과

μRTS 자원 수집에서 네 map 모두 r_episode 40을 기록했지만 solve time은 같지 않다

Metric	4 × 4	10 × 10	16 × 16	24 × 24
r_episode	40.00	40.00	40.00	40.00
t_solve	8.67%	11.13%	11.47%	18.38%
t_first	0.07%	0.05%	0.08%	0.07%

Masking은 네 map에서 작동했지만 보편적 scaling 정리는 아니다

Penalty 탐색의 scaling 한계

$r_{\text{invalid}} = -0.01$ 은 작은 map에서 작동했지만 큰 map에서는 첫 reward 탐색이 어려워졌다

조건	4×4	10×10	24×24	해석
Penalty r_{episode}	40.00	0.50	0.50	-0.01 penalty
Penalty t_{first}	-	1.57%	-	Table 2의 -0.01 행
Mask t_{first}	-	0.05%	-	같은 10×10 비교
Penalty - 1	학습 실패	학습 실패	학습 실패	강한 벌점이 exploration을 억제

이 실험은 penalty가 보편적으로 실패함이 아니라 reward scale 조정과 탐색 비용을 보여 준다

Naive update와 mask 제거 결과

Distribution mismatch와 deployment mismatch는 서로 다른 실패 경계를 보여 준다

조건	Metric	Map별 결과	읽는 법
Naive masking	24×24 t_solve	49.14%	Mask-consistent 18.38%보다 낮음
Naive masking	PPO KL	다른 실험보다 매우 높음	Update 안정성 경고
Mask 제거	r_episode	33.53 / 25.93 / 17.32 / 17.37	4, 10, 16, 24 map 순서
Mask 제거	a_null	63.57 / 128.76 / 165.12 / 150.06	각 map에서 invalid source 선택 재발

높은 return 일부가 gradient 정당성이나 mask 없는 배포의 안전성을 증명하지 않는다

실험의 partial mask

여덟 component 중 Source Unit과 Attack Target Unit만 mask하므로 전체 행동 validity는 보장되지 않는다

Component 집합	Mask 제공	남는 invalid 원인
Source Unit	yes	선택 가능한 unit만 남김
Attack Target Unit	yes	공격 가능한 target index를 제한
Action Type	no	현재 unit이 수행할 수 없는 type 가능
방향 parameter 4종	no	선택 type과 맞지 않는 방향 가능
Produce Type	no	생산 조건과 맞지 않는 종류 가능

Table 2의 masking 결과는 feature-complete constraint가 아니라 두 component mask의 결과다

Action factorization과 masking의 역할

Factorization은 출력 수를 줄이고 masking은 선택된 component의 feasibility를 제한한다

구성	Logit 또는 조합 수	성장	기능
Full joint categorical	$9216(hw)^2$	Map 면적 hw의 제곱	모든 조합을 한 분포로 표현
Square map $n \times n$	$9216n^4$	한 변 n의 4제곱	면적 기준 exponential이 아님
Factorized heads	$2hw + 36$	Map 면적에 선형	각 component의 logits를 따로 출력
Partial masking	Source와 Target subset	State validity에 따라 변함	알려진 invalid index를 제외
관계	함께 사용할 수 있음	서로 다른 병목을 다룸	Factorization 자체가 Prop. 1의 증명 아님

다른 환경에 적용할 조건

Masking을 옮길 때 validity의 출처와 sampling-loss 일치를 먼저 확인한다

확인 항목	필요한 조건	어긋날 때의 위험
Validity 공급자	환경 규칙 또는 검증된 constraint	틀린 valid action을 제거
상태 동기화	현재 상태의 최신 mask	지난 상태의 validity를 사용
Sampling과 loss	둘 다 같은 masked π'	Naive distribution mismatch
평가 interface	학습과 같은 mask 제공 여부 명시	배포 policy가 달라짐
Component coupling	조건부 parameter validity 처리	Partial mask 뒤 invalid 조합 잔존

Invalid action masking의 핵심

Support 제한, gradient 정합성, 실험 범위를 함께 봐야 논문의 결론을 정확히 사용할 수 있다

구분	정확한 결론	경계
Policy	Invalid logit을 치환한 π' 가 behavior policy	Finite M의 invalid 확률은 아주 작지만 0은 아님
Gradient	Sampling과 log-probability를 π' 로 맞추면 π' 의 policy gradient	Raw invalid-logit gradient는 치환 Jacobian 때문에 0
Penalty	Invalid sampling 뒤 reward를 바꾸는 별도 방법	Mask와 같은 objective나 탐색 경로가 아님
Evidence	μ RTS PPO의 partial mask에서 scaling 이점을 관찰	모든 constraint와 deployment를 보장하지 않음

같은 masked π' 가 sampling과 loss를 정의해야 한다